МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2 по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Д. М. Мишин
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Реализовать “камеру” с возможностью ее перемещения и освещение по модели Блинн-Фонга с использованием множества типов источников света.

Вариант 18. Сфера.

Метод решения

Сфера строится процедурно с плотной сеткой: 32 полосы и 64 сектора, то есть (32 + 1) · (64 + 1) = 2145 вершин и 32 · 64 · 6 = 12288 индексов. К позиции добавлен второй атрибут вершины – нормаль. Для сферы она совпадает с нормированным радиус-вектором: n = p / r. В сцену добавлен пол, плоскость из 4 вершин с нормалью (0, 1, 0).

Камера задаётся позицией, углами рыскания ψ и тангажа θ и углом обзора. Направление взгляда:

$$f = (\cos\theta \cdot \sin\psi,\ \sin\theta,\ -\cos\theta \cdot \cos\psi)$$

Матрица вида V = lookAt(pos, pos + f, up), матрица проекции P = perspective(fov, aspect, 0.1, 100). Правая кнопка мыши поворачивает камеру, клавиши W/S/A/D двигают её вдоль взгляда и вбок, Q/E – вниз и вверх. Тангаж ограничен диапазоном [−89°, 89°], чтобы взгляд не совпал с вектором up, а перемещение умножается на Δt и не зависит от частоты кадров.

Освещение считается во фрагментном шейдере в мировых координатах по модели Блинна-Фонга. Нормаль преобразуется матрицей $(M^{-1})^T$, чтобы оставаться перпендикулярной поверхности при неравномерном масштабировании. Для единичных векторов на источник L, на камеру V и нормали N вклад источника равен:

$$H = normalize(L + V),\quad I = (k_d \cdot max(N \cdot L, 0) + k_s \cdot max(N \cdot H, 0)^{\wedge}s) \cdot E$$

где k_d – альбедо, k_s – цвет блика, s – коэффициент блика, E – освещённость от источника. Итоговый цвет равен сумме фонового члена k_a · k_d и вкладов всех источников:

$$color = k_a \cdot k_d + \Sigma\, I_i$$

Поддержаны три типа источников:

- направленный – один, задаётся направлением, цветом и интенсивностью, без затухания;
- точечный – задаётся позицией, цветом и интенсивностью, освещённость убывает по закону обратных квадратов: $E = I / d^2$, где d – расстояние до источника (d^2 ограничено снизу значением 10^{-4}, чтобы не делить на ноль);

- прожекторный – то же затухание, плюс направление и два угла, внутренний и внешний. Если cos θ = dot(−L, dir), то множитель конуса равен smoothstep(cos θ_out, cos θ_in, cos θ): внутри внутреннего угла он равен 1, вне внешнего 0, а между ними плавно меняется, поэтому края мягкие. При внешнем угле не больше внутреннего граница получается резкой.

Данные в шейдеры передаются двумя наборами дескрипторов. Набор сцены содержит uniform-буфер с матрицей view-projection, позицией камеры, фоновым и направленным светом и числом источников, а также два storage-буфера с массивами точечных и прожекторных источников. Набор модели содержит uniform-буфер с матрицей модели, матрицей нормалей и параметрами материала; у сферы и пола он свой. Шейдер перебирает источники до заданного в uniform-буфере числа, поэтому максимальный размер массива в нём не фиксирован. Когда источников становится больше, чем помещается в storage-буфер, буфер пересоздаётся с удвоенным размером, а дескриптор обновляется. Источники добавляются и удаляются кнопками интерфейса.

Анимация та же, что в первой работе: фаза накапливается как $\psi = \psi + v \cdot \Delta t$, сфера движется по окружности с колебанием по высоте и вращается вокруг своей оси. Точечные источники можно заставить вращаться вокруг оси Y.

Общие объекты Vulkan: VkInstance, VkPhysicalDevice и VkDevice (подключение к видеоядру), VkQueue (очередь команд), VkSurfaceKHR и VkSwapchainKHR (вывод в окно), VkRenderPass и VkFramebuffer с вложениями цвета и глубины, VkCommandBuffer (команды кадра), VkSemaphore и VkFence (синхронизация), VmaAllocator (память). Для достижения цели работы созданы:

- вершинный и индексный VkBuffer для сферы и для пола (вершина 24 байта: позиция и нормаль);
- uniform-буферы сцены (144 байта) и двух моделей (по 160 байт), storage-буферы точечных (32 байта на источник) и прожекторных (64 байта на источник) источников;
- два VkDescriptorSetLayout (сцена и модель), VkDescriptorPool и три VkDescriptorSet;
- шейдеры lit.vert / lit.frag для освещённых объектов и marker.vert / marker.frag для маркеров источников, скомпилированные в SPIR-V;
- два VkPipeline: основной с двумя наборами дескрипторов и маркерный с push-константами в 80 байт (матрица модели и цвет); оба

с тестом глубины (VK_COMPARE_OP_LESS), отбрасыванием задних граней и динамическими областью вывода и прямоугольником отсечения.

Положение источников отмечено маленькими сферами цвета источника. Маркерный шейдер выдаёт постоянный цвет, поэтому они не освещаются. Каждый кадр записываются vkCmdBeginRenderPass, привязка pipeline и наборов дескрипторов, отрисовка сферы и пола через vkCmdDrawIndexed, отрисовка маркеров и vkCmdEndRenderPass. Параметры камеры, света, материалов и анимации задаются элементами ImGui, есть и кнопка сброса сцены.

Результаты

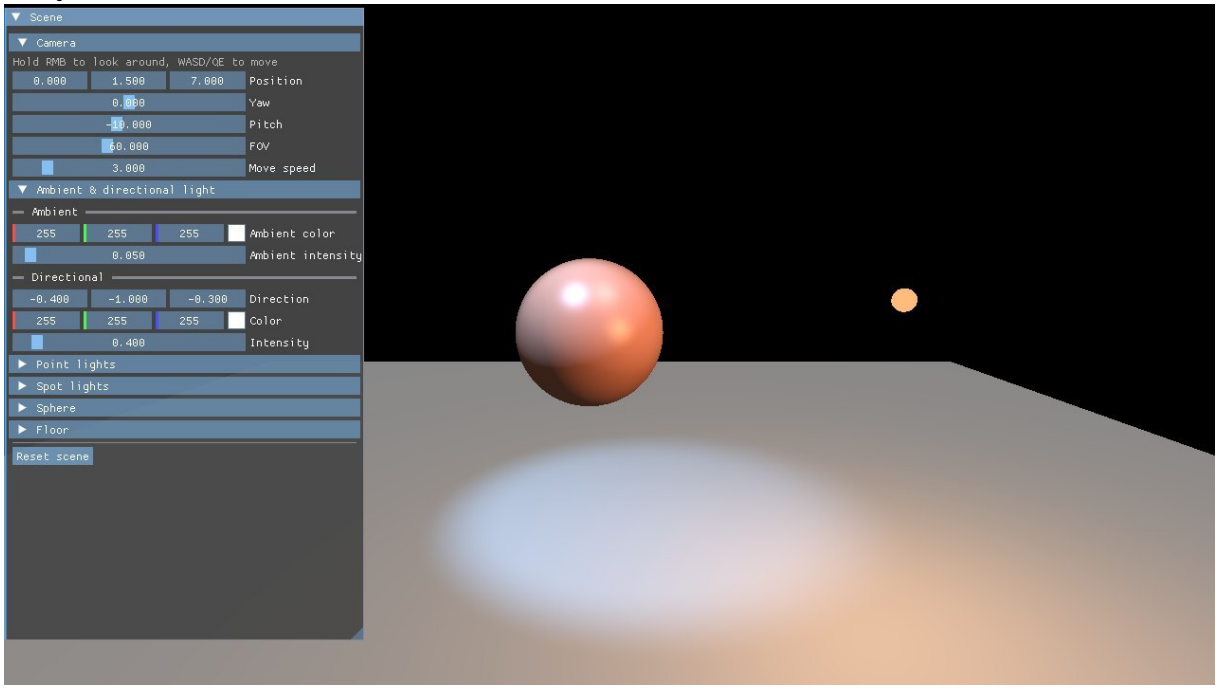


Рис. 1. Начальное состояние сферы.

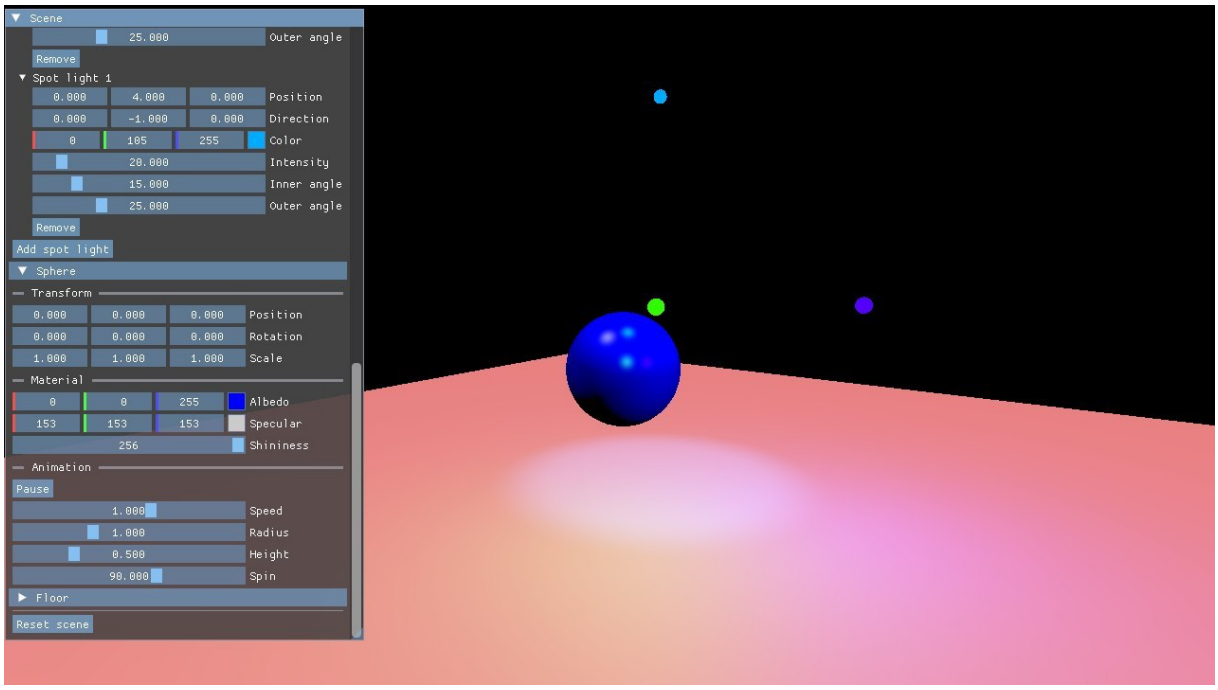


Рис. 2. Измененные параметры.

Выводы

В ходе работы я реализовал свободную камеру и освещение по модели Блинна-Фонга. Научился генерировать нормали процедурно и правильно преобразовывать их в мировое пространство. Реализовал три типа источников: направленный, точечные и прожекторные, для последних двух с затуханием по закону обратных квадратов и плавным краем конуса через smoothstep. Разобрался, как разделять данные по наборам дескрипторов и передавать в шейдер массивы произвольной длины через storage-буферы с их пересозданием при росте. Благодаря интерфейсу ImGui все параметры камеры, света и материалов можно менять в реальном времени и сразу видеть результат.